

Response to Code Review Methodologies Thread

The challenges described (self-review bias, memory/context pollution from prior sessions or MEMORY.md files, inconsistent quality from one-shot /code-review skills, token costs with GitLab MCP integration, and the desire for true “fresh eyes”) are exactly the problems I was facing about a year ago.

I solved them by building **vibe-palace**: a purpose-built MCP server paired with a durable **vault** — an Obsidian-compatible, git-backed repository that synchronizes cleanly across machines, team members, and multiple projects. All captured artifacts, session state, tasks, reviews, and learnings live in the vault instead of in host-local directories. Built-in semantic controls, surface gates, and conflict-prevention mechanisms keep the entire team sharing a single authoritative frame of reference for any given project.

The result is a system that delivers consistently superior adversarial reviews and high-velocity, high-confidence delivery. It has worked *incredibly* well in practice.

Vibe-palace moves the project continuity artifacts out of the harness and into an Obsidian-compatible vault of human- and machine-readable markdown documents. In many regards, vibe-palace becomes the harness regardless of which vendor provides the user interface.

The Harness Is What Matters

An LLM is fundamentally a next-token predictor. The quality gap between mediocre and exceptional output is almost entirely determined by the **harness** — what context it receives first, the steering instructions, the persistent memory model, and the workflow discipline that prevents inductive drift.

Early in the agent era (sub-120k token contexts), lightweight Agile-style iteration made sense. Today, with 500k+ token windows, reliable subagent orchestration, and a durable vault, the optimal pattern has shifted toward **spec-driven development + rigorous adversarial review**. This looks superficially like old-school Waterfall, but because one developer with the right tooling can now deliver in a day what used to take a 10-person team a month, the timeline gap between requirements and implementation has collapsed — removing the classic arguments for constant requirement churn.

The specification becomes the durable contract. The code is a reproducible artifact. The vault preserves the *why*, the *plan*, the *review findings*, and the *verification evidence* — not just the code.

Spec driven development and the hazards of inductive reasoning

With AI, spec-driven development makes the specification the binding contract to realize a product. The actual code that realizes it is almost irrelevant. I can take any well-written spec and re-implement the product in most any programming language at any time. The codebase at any point in time is really just a transitory and reproducible artifact *if* the spec is written correctly.

This is in stark contrast to Agile, where working code is the product and artifacts like documentation, integration, and unit tests are necessary byproducts needed to meet the definition of “done.”

As a human with very finite context, I still need agile disciplines to keep myself on target — but my agentic AI partners do not.

This particular article provided me with independent affirmation of where my intuition was leading me while researching my contribution to this thread: [Agile is Dead](#)

It’s a bit over stated, but the concepts are real. I am looking at integrating some of the ideas here to fill in the the gaps of vibe-palace. I do not agree with how it is implemented - at all, but I find it interesting that I independently developed much of the same scaffolding: [What is Spec-Driven Development?](#)

The hazards of inductive reasoning

LLMs only do one thing: predict the next token from a history of input tokens. The biggest differentiator between frontier models (and most advanced open-parameter models) is the harness that steers the

probabilistic path through all the possible token associations captured in the LLM's vast training data. At the beginning of any AI context thread, two things gate the goal-driven behavior of the outcome:

- The guardrails and content injection of the harness.
- The first user prompt offered in the first turn.

The goals of the rest of the session will include those two token sets, even as the iteration spans hundreds of turns. Deductive reasoning is nearly the opposite of this process, but it can be coaxed out of an LLM when it is deliberately driven in an adversarial role. Inductive reasoning looks at what has been and tries to predict what comes next. Left unchecked, it produces exactly the self-review blindness and context-pollution problems described in this thread.

How vibe-palace delivers true adversarial review

My system directly counters the failure modes raised in this thread:

- **Fresh, authoritative context on every restart, using minimal tokens**

Every session begins with `/vpc-restart`, which triggers `vp_bootstrap_context`. This delivers a clean index rather than large documents. I then explicitly fetch only what is needed via `vp_read_resource` on `resume.md` and `workflow.md`.

`resume.md` is deliberately kept thin — it serves primarily as an **index and gateway** into current work state, open tasks, and key threads. The actual detailed history lives in individual task/epic files and in `iterations.md`. This design lets me restore a ready-to-resume context in as few tokens as possible.

Because subordinate agents and sub-skills are not loaded with complex slash commands, large context histories, or unnecessary bloat, they can devote their full context window to the narrow task the orchestrating agent is supervising.

- **The vault as shared source of truth**

Unlike host-local `MEMORY.md` files that create per-user pollution and prevent fresh review, everything lives in the vault — a git-synchronized, Obsidian-compatible repository. It travels cleanly across machines, team members, and projects while built-in semantic controls, surface gates, and conflict-prevention mechanisms keep the shared frame of reference coherent and authoritative.

- **Investigation-first doctrine (never premature implementation)**

The doctrine (fetched on every restart) mandates plan mode for any non-trivial work. I do not allow agents to jump to code. Every significant change produces a written plan that lives in the vault via `vp_manage_task`.

- **Structured adversarial gates before execution**

- `vpc-review-plan`: Critical architecture review of the plan *before* any code is touched.
- `vps-pair-reviewer`: A dedicated “review chair” agent that represents my architectural interests while another agent implements. This creates genuine adversarial tension.
- `vps-code-digger`: A standing improvement loop (blind run → oracle validation against real project state → proposed edits). It never retires — it continuously audits and improves the tools and skills.

- **Herdr for parallel, isolated panes**

I use [Herdr](#) to run dedicated reviewer, implementer, and auditor panes in the same workspace with clean context separation.

- **Immutable discipline (“nothing is done until the human says so”)**

I use `vpc-execute-plan` (which runs in an isolated git worktree), followed by personal review, `vpc-stage`, `commit`, and `vpc-wrap`. This enforces accountability and prevents premature closure.

These mechanisms give the equivalent of “fresh eyes from a completely different account” without needing a separate paid account or fighting with GitLab MCP token issues. The combination of harness, vault, and doctrine creates reliable adversarial separation.

Results and daily workflow

This system has allowed me to tackle extremely complex refactors (legacy task header migration, surface gating at the dispatch seam, fence-aware classifiers, etc.) with near-zero regression. Reviews produced by the `vps-pair-reviewer` + `vps-code-digger` combination routinely catch issues that self-review (even with 1M context models) misses.

My daily flow:

1. `/vpc-restart` — full context restoration (doctrine + resume + workflow).
2. `vpc-review-plan` on any non-trivial task or MR.
3. Parallel Herdr panes: one for implementation, one for `vps-pair-reviewer`.
4. `vps-code-digger` for deep adversarial audit.
5. `vpc-execute-plan` in isolated worktree.
6. Personal review → `vpc-stage` → commit → `vpc-wrap`.

I would be happy to share the embedded doctrine, the specific skills (`vps-code-digger`, `vps-pair-reviewer`), or run a live demo on one of your branches. The entire system is open source at github.com/suykerbuyk/vibe-palace (dual MIT/Apache-2.0 licensing) and the vault + workflow patterns are reusable across projects.

Happy to discuss further.

- John “S”